

nRF Connect SDK - 2.9.2

Contents

Zigbee shell 3

1. Zigbee shell

Note

Support for Zigbee R22 has been deprecated in nRF Connect SDK v2.8.0 and it will be removed in one of the future releases, following the [deprecation policy](#). As a result, the nRF52833, nRF52840, and nRF5340 SoCs are not recommended for new Zigbee designs. Experimental support for Zigbee R23 is available for the nRF54L Series as an [nRF Connect SDK Add-on](#).

The Zigbee shell library implements a set of Zigbee shell commands that allow you to control the device behavior and perform operations in the Zigbee network, such as steering the network, discovering devices, and reading the network properties. Enabling a device to work with these commands simplifies testing and debugging of an existing network and reduces the amount of reprogramming operations.

Configuration

The Zigbee shell commands are implemented using Zephyr's [Shell](#) interface. By default, Zephyr shell uses the UART backend. To change this and other Zephyr's shell settings (for example, the prompt or the maximum amount of accepted command arguments), read the documentation page in Zephyr.

You can add support for Zigbee shell commands to any of the available [Zigbee samples](#), except for the [NCP sample](#). Some of the commands use an endpoint for sending packets, so no endpoint handler is allowed to be registered for this endpoint.

To configure the Zigbee shell library, use the following options:

- `CONFIG_ZIGBEE_SHELL`
- `CONFIG_ZIGBEE_SHELL_ENDPOINT`
- `CONFIG_ZIGBEE_SHELL_DEBUG_CMD`
- `CONFIG_ZIGBEE_SHELL_LOG_LEVEL`
- `CONFIG_ZIGBEE_SHELL_CTX_MGR_ENTRIES_NBR`
- `CONFIG_ZIGBEE_SHELL_MONITOR_CACHE_SIZE`
- `CONFIG_ZIGBEE_SHELL_ZCL_CMD_TIMEOUT`

For detailed steps about configuring the library in a Zigbee sample or application, see [Configuring Zigbee endpoint logger](#).

Supported backends

Zigbee shell commands are available for the following backends when testing samples:

- UART
- Segger RTT

You can run the Zigbee shell commands after connecting and configuring the backend for testing. For more information, see [Testing and optimization](#).

Zigbee shell command list

This section lists commands that are supported by [Zigbee samples](#).

Description convention

Every command prints `Done` when it is finished, or an error with the reason why it occurs.

The command argument description uses the following convention:

- Square brackets mean that an argument is optional:

```
command [arg]
```

- A single letter before an argument name defines the format of the argument:

```
command d:arg1 h:arg2
```

- *h* stands for hexadecimal strings.
- *d* stands for decimal values.
- The ellipsis after an argument means that the preceding argument can be repeated several times:

```
command arg ...
```

help

Display help for all available shell commands.

```
[group] help
```

If the optional argument is not provided, displays help for all command groups.

If the optional argument is provided, displays help for subcommands of the specified command group. For example, `zdo help` displays help for all `zdo` commands.

Example:

```
help
Please press the <Tab> button to see all available commands.
You can also use the <Tab> button to prompt or auto-complete all commands or its subcommand
```

ds.

You can **try** to call commands **with** `<-h>` **or** `<--help>` parameter **for** more information.

Shell supports following meta-keys:

Ctrl + (a key from: abcdefklmpuw)

Alt + (a key from: bf)

Please refer to shell documentation **for** more details.

Available commands:

bdb	:Base device behaviour manipulation.
clear	:Clear screen.
debug	:Return state of debug mode.
device	:Device commands
devmem	:Read/write physical memory" <code>devmem address [width [value]]</code> "
flash	:Flash shell commands
help	:Prints the help message.
history	:Command history.
kernel	:Kernel commands
nbr	:Zigbee neighbor table.
nrf_clock_control	:Clock control commands
nvrnm	:Zigbee NVRAM manipulation.
resize	:Console gets terminal screen size or assumes default in case the readout fails. It must be executed after each terminal width change to ensure correct text display.
sensor	:Sensor commands
shell	:Useful, not Unix-like shell commands.
version	:Print firmware version
zcl	:ZCL subsystem commands.
zdo	:ZDO manipulation.
zscheduler	:Zigbee scheduler manipulation.

bdb role

Set or get the Zigbee role of a device.

```
bdb role [role]
```

Note

Setting only before **bdb start**. Reading only after **bdb start**.

If the optional argument is not provided, get the state of the device. Returns the following values:

- **zc** if it is a coordinator.

- `zr` if it is a router.
- `zed` if it is an end device.

If the optional argument is provided, set the device role to *role*. Can be either `zc` or `zr`.

Note

Zigbee End Device is not currently supported by the Shell sample.

bdb extpanid

Set or get the Zigbee Extended Pan ID value.

```
bdb extpanid [h:id]
```

Note

Setting only before `bdb start`. Reading only after `bdb start`.

If the optional argument is not provided, gets the extended PAN ID of the joined network.

If the optional argument is provided, gets the extended PAN ID to *id*.

bdb panid

Set or get the Zigbee PAN ID value.

```
bdb panid [h:id]
```

Note

Setting only before `bdb start`. Reading only after `bdb start`.

If the optional argument is not provided, gets the PAN ID of the joined network. If the optional argument is provided, sets the PAN ID to *id*.

bdb start

Start the commissioning process.

```
> bdb start
Started coordinator
Done
```

bdb channel

Set or get the 802.15.4 channel.

```
bdb channel n
```

Note

Setting only before **bdb start**.

If the optional argument is not provided, get the current number and bitmask of the channel.

If the optional argument is provided:

- If *n* is in `[11:26]` range, set to that channel.
- Otherwise, treat *n* as bitmask (logical or of a single bit shifted by channel number).

Example:

```
> bdb channel 0x110000
Setting channel bitmask to 110000
Done
```

bdb ic add

Add information about the install code on the trust center.

```
bdb ic add h:install code h:eui64
```

For *h:eui64*, use the address of the joining device. For *h:install code*, use 128bit install code with correct CRC value.

Note

Adding install codes only after **bdb start**.

Provide the install code as an ASCII-encoded hexadecimal file that includes CRC16/X-25 in little-endian order.

For production devices, make sure that an install code is installed by the production configuration present in the flash.

Must only be used on a coordinator.

Example:

```
> bdb ic add 83FED3407A939723A5C639B26916D505C3B5 0B010E2F79E9DBFA
Done
```

bdb ic list

Read and print install codes stored on the device.

```
bdb ic list
```

Note

Use only after `bdb start`.

For production devices, make sure that an install code is installed by the production configuration present in the flash.

Must only be used on a coordinator.

Example:

```
> bdb ic list
[idx] EUI64:          IC:          options:
[ 0] 0b010e2f79e9dbfa 83fed3407a939723a5c639b26916d505c3b5 0x3
Total entries for the install codes table: 1
Done
```

bdb ic policy

Set the trust center install code policy.

```
bdb ic policy enable|disable
```

Note

For production devices, make sure that an install code is installed by the production configuration present in the flash.

Must only be used on a coordinator.

Example:

```
> bdb ic policy enable
Done
```

bdb ic set

Set install code on the device.

```
bdb ic set h:install code
```


Must only be used on a joining device.

Note

Setting only before `bdb start`.

Provide the install code as an ASCII-encoded hexadecimal file that includes CRC16/X-25 in little-endian order.

For production devices, make sure that an install code is installed by the production configuration present in the flash.

Example:

```
> bdb ic set 83fed3407a939723a5c639b26916d505c3b5
Done
```

bdb legacy

Enable or disable the legacy device support.

```
bdb legacy enable|disable
```

Allow or disallow legacy pre-r21 devices on the Zigbee network.

Example:

```
> bdb legacy enable
Done
```

bdb nwkey

Set network key.

```
bdb nwkey h:key
```

Set a pre-defined network key *key* instead of a random one.

Note

Setting only before `bdb start`.

Example:

```
> bdb nwkey 00112233445566778899aabbccddeeff
Done
```

bdb factory_reset

Perform a factory reset using local action. See Base Device Behavior specification chapter 9.5 for details.

```
> bdb factory_reset
Done
```

bdb child_max

Set the amount of child devices that is equal to *d:nbr*.

```
> bdb child_max d:nbr
```

Note

Setting only before **bdb** start.

Example:

```
> bdb child_max 16
Setting max children to: 16
Done
```

zcl cmd

Send a generic ZCL command to the remote node.

```
zcl cmd [-d] h:dst_addr d:ep h:cluster [-p h:profile] h:cmd_ID [-l h:payload]
```

Note

By default, the profile is set to Home Automation Profile, and the payload is empty.

The payload requires the **little-endian** byte order.

To send a request using binding table entries, set **dst_addr** and **ep** to **0**. To send as groupcast, set **dst_addr** to a group address and **ep** to **0**.

Send a generic ZCL command with ID **cmd_ID** and payload **payload** to the cluster **cluster**. The cluster belongs to the profile **profile**, which resides on the endpoint **ep** of the remote node **dst_addr**. Optional default response can be requested with **-d**.

Examples:

```
zcl cmd 0x1234 10 0x0006 0x00
```

This command sends the Off command from the On/Off cluster (ZCL specification 3.8.2.3.1) to the device with the short address `0x1234` and endpoint `10`.

```
zcl cmd 0x1234 10 0x0008 0x00 -1 FF0A00
```

This command sends the Move to Level command from the Level Control cluster (ZCL specification 3.10.2.3.1) to the device with the short address `0x1234` and endpoint `10`, asking it to move the CurrentLevel attribute to a new value `255` in 1 second.

```
zcl cmd -d 0x1234 10 0x0008 -p 0x0104 0x00 -1 FF0A00
```

This command sends the same Move to Level command and requests additional Default response. The same Home Automation Profile is used, but is set directly instead.

zcl attr read

Retrieve the attribute value of the remote node.

```
zcl attr read h:dst_addr d:ep h:cluster [-c] h:profile h:attr_id
```

Read the value of the attribute `attr_id` in the cluster `cluster`. The cluster belongs to the profile `profile`, which resides on the endpoint `ep` of the remote node `dst_addr`. If the attribute is on the client role side of the cluster, use the `-c` switch.

Example:

```
> zcl attr read 0x1234 10 0x0000 0x0104 0x00
ID: 0 Type: 20 Value: 3
Done
```

This command sends the Read Attributes command (ZCL specification 2.5.1) to the device with the short address `0x1234` and endpoint `10`, asking it to reply with the ZCLVersion attribute value of the Basic cluster.

zcl attr write

Write the attribute value to the remote node.

```
zcl attr write h:dst_addr d:ep h:cluster [-c] h:profile h:attr_id h:attr_type h:attr_value
```

Write the `attr_value` value of the attribute `attr_id` of the type `attr_type` in the cluster `cluster`. The cluster belongs to the profile `profile`, which resides on the endpoint `ep` of the remote node `dst_addr`. If the attribute is on the client role side of the cluster, use the `-c` switch.

Note

The `attr_value` value must be in the hexadecimal format, unless it is a string (`attr_type == 42`), then it must be a string.

Example:

```
> zcl attr write 0x1234 10 0x0003 0x0104 0x00 0x21 0x0F
Done
```

This command sends the Write Attributes command (ZCL specification 2.5.3) to the device with the short address `0x1234` and endpoint `10`, asking it to set the IdentifyTime attribute value to `15` in the identify cluster.

zcl subscribe on

Subscribe to the attribute changes on the remote node.

```
zcl subscribe on h:addr d:ep h:cluster h:profile h:attr_id d:attr_type [d:min interval
(s)] [d:max interval (s)]
```

Enable reporting on the node identified by the address `addr`, with the endpoint `ep` that uses the profile `profile` of the attribute `attr_id` with the type `attr_type` in the cluster `cluster`.

The reports must be generated in intervals not shorter than `min interval` (1 second by default) and not longer than `max interval` (60 seconds by default).

Example:

```
> zcl subscribe on 0x1234 10 0x0006 0x0104 0x00 16 5 20
Done
```

This command sends the Configure Reporting command (ZCL specification 2.5.7) to the device with the short address `0x1234` and endpoint `10`, asking it to configure reporting for the OnOff attribute of the On/Off cluster with minimum reporting interval of `5` seconds and maximum reporting interval of `20` seconds.

zcl subscribe off

Unsubscribe from the attribute reports.

```
zcl subscribe off h:addr d:ep h:cluster h:profile h:attr_id d:attr_type
```

Disable reporting on the node identified by the address `addr`, with the endpoint `ep` that uses the profile `profile` of the attribute `attr_id` with the type `attr_type` in the cluster `cluster`.

Example:

```
> zcl subscribe off 0x1234 10 0x0006 0x0104 0x00 16
Done
```

This command sends the Configure Reporting command (ZCL specification 2.5.7) to the device with the short address `0x1234` and endpoint `10`, asking it to stop issuing reports for the On/Off attribute of the On/Off cluster, by setting maximum reporting interval to `0xffff`.

zcl ping

Ping other devices using ZCL.

```
zcl ping [--no-echo] [--aps-ack] h:dst_addr d:payload_size
```

Example:

```
zcl ping 0b010eaafd745dfa 32
```

Note

Use only after `bdb start`.

Issue a ping-style shell command to another Shell node with the given 16-bit destination address (*dst_addr*) by using a payload equal to *payload_size* bytes. The command is sent and received on endpoints with the same ID.

This shell command uses a custom ZCL frame, which is constructed as a ZCL frame of a custom ping ZCL cluster with the cluster ID `0xBEEF`. For details, see the implementation of `ping_request_send()` in `subsys/zigbee/lib/zigbee_shell/src/zigbee_shell_cmd_ping.c`.

The command measures the time needed for a Zigbee frame to travel between two nodes in the network (there and back again). The shell command sends a ping request ZCL command, which is followed by a ping reply ZCL command. The IDs of the ping request change depending on optional arguments. The ping reply ID stays the same (`0x01`).

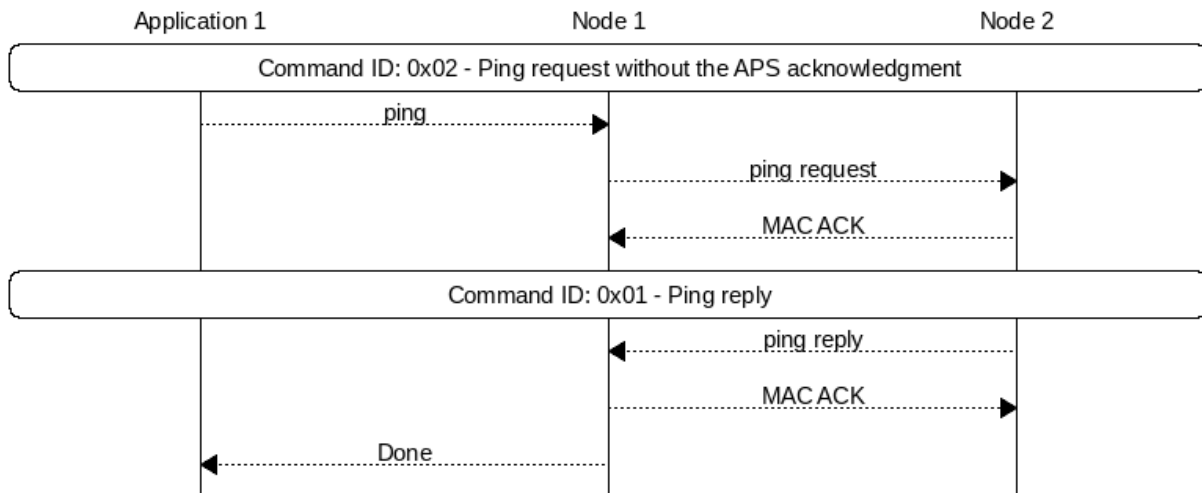
The following optional argument are available:

- `--aps-ack` requests an APS acknowledgment
- `--no-echo` asks the destination node not to send the ping reply

Both arguments can be used at the same time. See the following graphs for use cases.

Case 1: Ping with echo, but without the APS acknowledgment

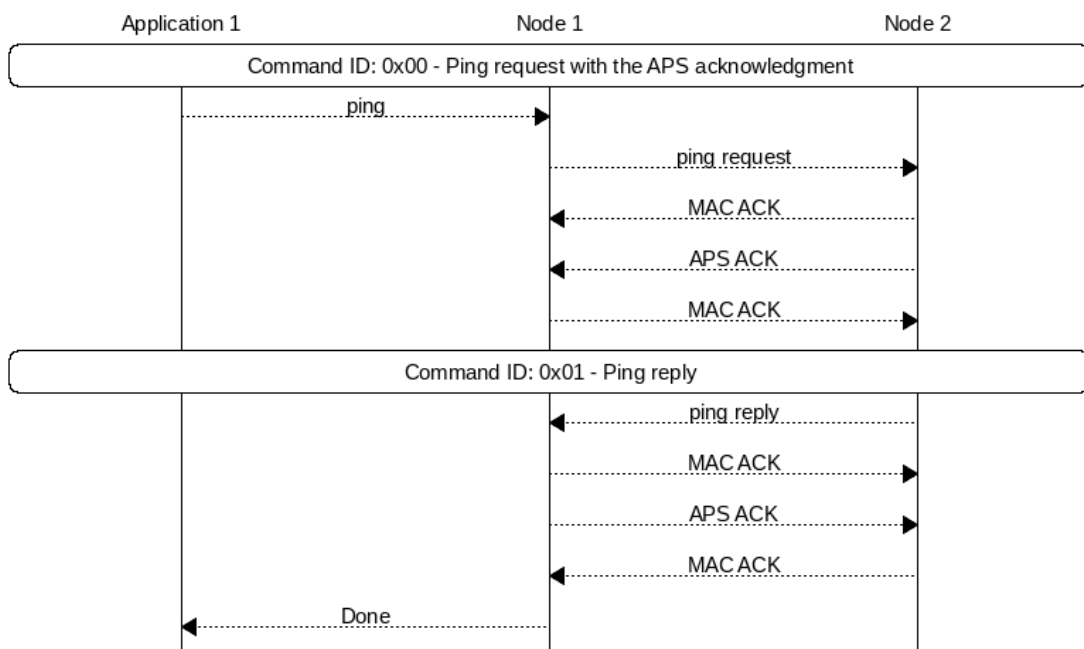
This is the default case, without optional arguments.



In this default case, the `zcl ping` command measures the time between sending the ping request and receiving the ping reply.

Case 2: Ping with echo and with the APS acknowledgment

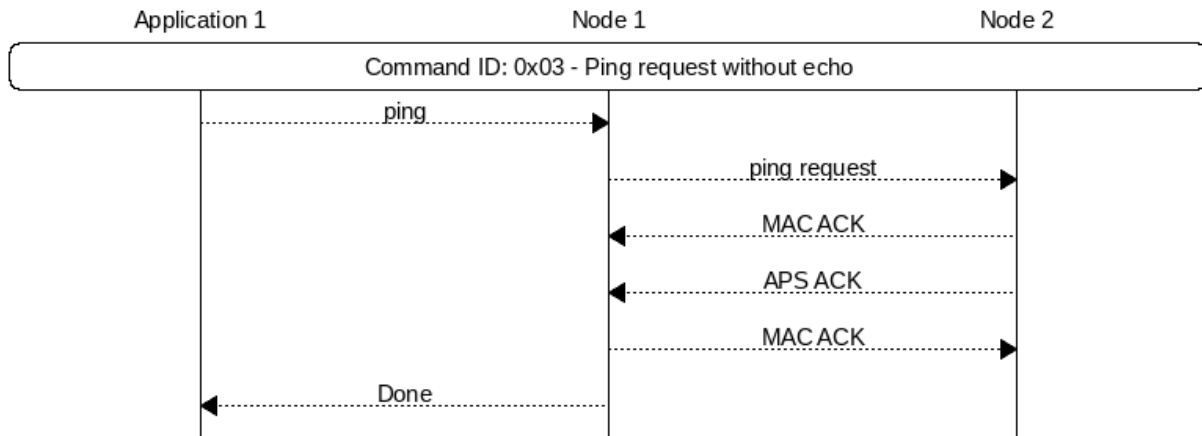
This is a case with the `--aps-ack` optional argument.



In this case, the `zcl ping` command measures the time between sending the ping request and receiving the ping reply.

Case 3: Ping without echo, but with the APS acknowledgment

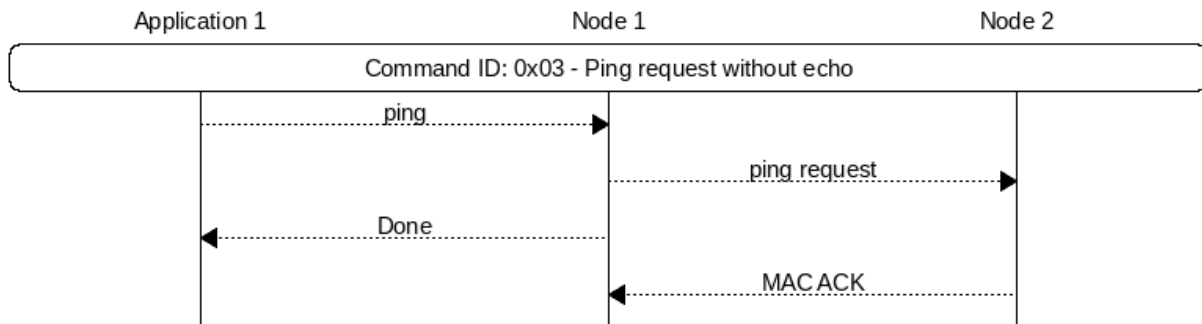
This is a case with both optional arguments provided, `--aps-ack` and `--no-echo`.



In this case, the `zcl ping` command measures the time between sending the ping request and receiving the APS acknowledgment.

Case 4: Ping without echo and without the APS acknowledgment

This is a case with the `--no-echo` optional argument.



In this case, the `zcl ping` command does not measure time after sending the ping request.

zcl groups add

Add the endpoint on the remote node to the group.

```
zcl groups add h:dst_addr d:ep [-p h:profile] h:group_id
```

Send Add Group command to the endpoint `ep` of the remote node `dst_addr` and add it to the group `group_id`. Providing profile ID `profile` is optional (Home Automation Profile is used by default). This command can be sent as groupcast. To send the command as groupcast, set `dst_addr` to a group address and `ep` to `0`.

Example 1:

```
> zcl groups add 0x1234 10 0x4321
```

```
Response command received, group: 0x4321, status: 0
Done
```

This command sends the Add Group command (ZCL specification 3.6.2.3.2) to the device with the short address `0x1234` and endpoint `10`, asking it to join the group `0x4321`.

Example 2:

```
> zcl groups add 0x1234 10 -p 0x0104 0x3210
Response command received, group: 0x3210, status: 0
Done
```

This command sends the Add Group command (ZCL specification 3.6.2.3.2) to the device with the short address `0x1234` and endpoint `10` with profile set to `0x0104`, asking it to join the group `0x3210`.

Example 3:

```
> zcl groups add 0x3210 0 0x1000
Done
```

This command sends the Add Group command (ZCL specification 3.6.2.3.2) as groupcast to devices that belong to the group `0x3210`, asking them to join the group `0x1000`.

zcl groups add_if_identify

Add the endpoints on the remote nodes to the group if the endpoints are in the Identify mode.

```
zcl groups add_if_identify h:dst_addr d:ep [-p h:profile] h:group_id
```

Send Add Group If Identifying command to the endpoint `ep` of the remote node `dst_addr` and add it to the group `group_id` if the endpoint on the remote node is in the Identify mode. Providing profile ID `profile` is optional (Home Automation Profile is used by default). This command can be sent as groupcast. To send the command as groupcast, set `dst_addr` to a group address and `ep` to `0`.

Example 1:

```
> zcl groups add_if_identify 0x1234 10 0x4321
Done
```

This command sends the Add Group If Identifying command (ZCL specification 3.6.2.3.7) to the device with the short address `0x1234` and endpoint `10`, asking it to join the group `0x4321` if is in the Identify mode.

Example 2:


```
> zcl groups add_if_identify 0x1234 10 -p 0x0104 0x3210
Done
```

This command sends the Add Group If Identifying command (ZCL specification 3.6.2.3.7) to the device with the short address `0x1234` and endpoint `10` with profile set to `0x0104`, asking it to join the group `0x3210` if it is in the Identify mode.

Example 3:

```
> zcl groups add_if_identify 0x3210 0 0x1000
Done
```

This command sends the Add Group If Identifying command (ZCL specification 3.6.2.3.7) as groupcast to devices that belong to the group `0x3210`, asking them to join the group `0x1000` if they are in the Identify mode.

zcl groups get_member

Get group membership from the endpoint on the remote node.

```
zcl groups get_member h:dst_addr d:ep [-p h:profile] [h:group IDs ...]
```

Send Get Group Membership command to the endpoint `ep` of the remote node `dst_addr` to get which groups from the provided group list are including the endpoint on the remote node. If no group ID is provided, the remote node is requested to response with information about all groups it is part of. Providing profile ID `profile` is optional (Home Automation Profile is used by default). This command can be sent as groupcast. To send the command as groupcast, set `dst_addr` to a group address and `ep` to `0`. Response from the remote node is parsed and printed only if the command was sent as unicast.

Example 1:

```
> zcl groups get_member 0x1234 10
short_addr=0x1234 ep=10 capacity=13 group_cnt=3 group_list=0x4321,0x3210,0x1000,
Done
```

This command sends the Get Group Membership command (ZCL specification 3.6.2.3.4) to the device with the short address `0x1234` and endpoint `10`, asking it to respond with the list of groups that include the device.

Example 2:

```
> zcl groups get_member 0x1234 10 -p 0x0104 0x4321 0x3210
short_addr=0x1234 ep=10 capacity=13 group_cnt=3 group_list=0x4321,0x3210,
Done
```

This command sends the Get Group Membership command (ZCL specification 3.6.2.3.4) to the device with the short address `0x1234` and endpoint `10` with profile set to `0x0104`, asking it to respond with the list of groups that include the device and that are present in the group list provided in the command.

zcl groups remove

Remove the endpoint on the remote node from the group.

```
zcl groups remove h:dst_addr d:ep [-p h:profile] h:group_id
```

Send Remove Group command to the endpoint `ep` of the remote node `dst_addr` and remove it from the group `group_id`. Providing profile ID `profile` is optional (Home Automation Profile is used by default). This command can be sent as groupcast. To send the command as groupcast, set `dst_addr` to a group address and `ep` to `0`.

Example 1:

```
> zcl groups remove 0x1234 10 0x4321
Response command received, group: 0x4321, status: 0
Done
```

This command sends the Remove Group command (ZCL specification 3.6.2.3.5) to the device with the short address `0x1234` and endpoint `10`, asking it to remove itself from the group `0x4321`.

Example 2:

```
> zcl groups remove 0x1234 10 -p 0x0104 0x3210
Response command received, group: 0x3210, status: 0
Done
```

This command sends the Remove Group command (ZCL specification 3.6.2.3.5) to the device with the short address `0x1234` and endpoint `10` with profile set to `0x0104`, asking it to remove itself from the group `0x3210`.

Example 3:

```
> zcl groups remove 0x3210 0 0x1000
Done
```

This command sends the Remove Group command (ZCL specification 3.6.2.3.5) as groupcast to devices that belong to the group `0x3210`, asking them to remove themselves from the group `0x1000`.

zcl groups remove_all

Remove the endpoint on the remote node from all groups.

```
zcl groups remove_all h:dst_addr d:ep [-p h:profile]
```

Send Remove All Groups command to the endpoint `ep` to the remote node `dst_addr`. Providing profile ID `profile` is optional (Home Automation Profile is used by default). This command can be sent as groupcast. To send the command as

groupcast, set `dst_addr` to a group address and `ep` to `0`.

Example 1:

```
> zcl groups remove_all 0x1234 10
Done
```

This command sends the Remove All Groups command (ZCL specification 3.6.2.3.6) to the device with the short address `0x1234` and endpoint `10`, asking it to remove itself from all groups.

Example 2:

```
> zcl groups remove_all 0x1234 10 -p 0x0104
Done
```

This command sends the Remove All Groups command (ZCL specification 3.6.2.3.6) to the device with the short address `0x1234` and endpoint `10` with profile set to `0x0104`, asking it to remove itself from all groups.

Example 3:

```
> zcl groups remove_all 0x1000 0
Done
```

This command sends the Remove All Groups command (ZCL specification 3.6.2.3.6) as groupcast to devices that belong to the group `0x3210`, asking them to remove themselves from all groups.

zdo simple_desc_req

Send Simple Descriptor Request.

```
zdo simple_desc_req h:dst_addr d:ep
```

Send Simple Descriptor Request to the given 16-bit destination address of the node (`dst_addr`) and the endpoint `ep`.

Example:

```
> zdo simple_desc_req 0xefba 10
src_addr=0xEFBA ep=0x260 profile_id=0x0102 app_dev_id=0x0 app_dev_ver=0x5
in_clusters=0x0000,0x0003,0x0004,0x0005,0x0006,0x0008,0x0300 out_clusters=0x0300
Done
```

zdo active_ep

Send Active Endpoint Request.

```
zdo active_ep h:dst_addr
```

Send Active Endpoint Request to the 16-bit destination address of the node (*dst_addr*).

Example:

```
> zdo active_ep 0xb4fc
> src_addr=B4FC ep=10,11,12
Done
```

zdo match_desc

Send match descriptor request.

```
zdo match_desc h:dst_addr
               h:req_addr h:prof_id
               d:n_input_clusters [h:input cluster IDs ...]
               d:n_output_clusters [h:output cluster IDs ...]
```

Send Match Descriptor Request to the 16-bit destination address of the node (*dst_addr*) that is a query about the requested address/type node (*req_addr*) of the *prof_id* profile ID, which must have at least one of input clusters (*n_input_clusters*), whose IDs are listed in [...], or at least one of output clusters (*n_output_clusters*), whose IDs are listed in [...]. The IDs can be either decimal values or hexadecimal strings.

Example:

```
match_desc 0xffffd 0xffffd 0x0104 1 6 0
```

In this example, the command sends a Match Descriptor Request to all non-sleeping nodes regarding all non-sleeping nodes that have 1 input cluster ON/OFF (ID 6) and 0 output clusters.

zdo bind on

Create a binding between two endpoints on two nodes.

```
zdo bind on h:source_eui64 d:source_ep h:dst_addr
           d:dst_ep h:source_cluster_id h:request_dst_addr
```

Note

To bind the device to a group address, set *dst_addr* to a group address and *dst_ep* to 0.

Create bound connection between a device identified by *source_eui64* and endpoint *source_ep*, and a device identified by

destination address *dst_addr* and destination endpoint *dst_ep*. The connection is created for ZCL commands and attributes assigned to the ZCL cluster *source_cluster_id* on the *request_dst_addr* node (usually short address corresponding to *source_eui64* argument).

Example:

```
zdo bind on 0B010E0405060708 1 0B010E4050607080 2 8
```

zdo bind off

Remove a binding between two endpoints on two nodes.

```
zdo bind off h:source_eui64 d:source_ep h:dst_eui64
            d:destination_ep h:source_cluster_id h:request_dst_addr
```

Note

To unbind the device from a group address, set *dst_addr* to a group address and *dst_ep* to 0.

Remove bound connection between a device identified by *source_eui64* and endpoint *source_ep*, and a device identified by destination address *dst_eui64* and destination endpoint *dst_ep*. The connection is removed for ZCL commands and attributes assigned to the ZCL cluster *source_cluster_id* on the *request_dst_addr* node (usually, the same address as for the *source_eui64* device).

zdo mgmt_bind

Read the binding table from a node.

```
zdo mgmt_bind h:dst_addr [d:start_index]
```

Send a request to the remote device identified by the 16-bit destination address (*dst_addr*) to read the binding table through *zdo mgmt_bind_req* (see spec. 2.4.3.3.4). If the whole binding table does not fit into a single *mgmt_bind_resp frame*, the request initiates a series of *mgmt_bind_req* requests to perform the full download of the binding table. *start_index* is the index of the first entry in the binding table where the reading starts. It is zero by default.

Example:

```
zdo mgmt_bind 0x1234
```

This command sends *mgmt_bind_req* to the device with short address *0x1234*, asking it to return its binding table.

Sample output:

```
[idx] src_address      src_endp cluster_id dst_addr_mode dst_addr      dst_endp
```

```
[ 0] 0b010ef8872c633e      10      0x0402          3 0b010e21591eef3e      64
[ 1] 0b010ef8872c633e      10      0x0403          3 0b010e21591eef3e      64
Total entries for the binding table: 2
Done
```

zdo mgmt_lqi

Send a ZDO Mgmt_Lqi_Req command to a remote device with the short address *short*.

```
zdo mgmt_lqi h:short [d:start_index]
```

start_index is the index of the first entry in the binding table where the reading starts. It is zero by default.

Example:

```
zdo mgmt_lqi 0x1234
```

This command sends `mgmt_lqi_req` to the device with short address `0x1234`, asking it to return its neighbor table.

zdo nwk_addr

Resolve the EUI64 address *eui64* to a short network address.

```
zdo nwk_addr h:eui64
```

Example:

```
zdo nwk_addr 0B010E0405060708
```

zdo ieee_addr

Resolve the EUI64 address *short_addr* by sending the IEEE address request.

```
zdo ieee_addr h:short_addr
```

zdo eui64

Get the EUI64 address of the Zigbee device.

```
> zdo eui64
0b010eaafd745dfa
Done
```

zdo short

Get the short 16-bit address of the Zigbee device.

```
> zdo short
0000
Done
```

zdo mgmt_leave

Send a request to a remote device to leave the network through `zdo mgmt_leave_req` (see the specification section 2.4.3.3.5).

```
zdo mgmt_leave h:dst_addr [h:device_address] [--children] [--rejoin]
```

Send `mgmt_leave_req` to a remote node specified by 16-bit destination address `dst_addr`. If the EUI64 `device_address` is omitted or it has a value equal to `0000000000000000`, the remote device at address `dst_addr` will remove itself from the network. If `device_address` has other value, it must be a long address corresponding to `dst_addr` or a long address of child node of `dst_addr`. The request is sent with *Remove Children* and *Rejoin* flags set to `0` by default. Use options `\--children` or `\--rejoin` to change the respective flags to `1`. For more details, see the section 2.4.3.3.5 of the specification.

Examples:

```
zdo mgmt_leave 0x1234
```

This command sends `mgmt_leave_req` to the device with the short address `0x1234`, asking it to remove itself from the network.

```
zdo mgmt_leave 0x1234 --rejoin
```

This command sends `mgmt_leave_req` to the device with the short address `0x1234`, asking it to remove itself from the network and perform rejoin.

```
zdo mgmt_leave 0x1234 0b010ef8872c633e
```

This command sends `mgmt_leave_req` to the device with the short address `0x1234`, asking it to remove device `0b010ef8872c633e` from the network. If the target device with the short address `0x1234` also has a long address `0b010ef8872c633e`, it will remove itself from the network. If the target device with the short address `0x1234` has a child with long address `0b010ef8872c633e`, it will remove the child from the network.

```
zdo mgmt_leave 0x1234 --children
```

This command sends `mgmt_leave_req` to the device with the short address `0x1234`, asking it to remove itself and all its

children from the network.

version

Print the firmware version.

```
version
```

Example:

```
> version
Shell: Sep  3 2020 13:34:28
ZBOSS: 3.3.0.2
Zephyr kernel version: 2.3.99
Done
```

debug

Enable or disable the debug mode in the Shell.

```
debug on|off
```

This command unblocks several additional commands in the Shell.

Note

When used, the additional commands can render the device unstable.

zscheduler suspend

Suspend Zigbee scheduler processing.

```
zscheduler suspend
```

Note

Use only after [bdb start](#).

zscheduler resume

Resume Zigbee scheduler processing.

```
zscheduler resume
```


Note

Use only after [bdb start](#).

nbr monitor on

Start monitoring the list of active Zigbee neighbors.

```
nbr monitor on
```

nbr monitor off

Stop monitoring the list of active Zigbee neighbors.

```
nbr monitor off
```

nbr monitor trigger

Trigger logging the list of active Zigbee neighbors.

Note

This function will trigger logging only if the Zigbee active neighbor monitor is started (see [nbr monitor on](#)).

```
nbr monitor trigger
```

nvrn enable

Enable Zigbee NVRAM.

Note

Setting only before [bdb start](#).

```
nvrn enable
```

nvrn disable

Disable Zigbee NVRAM.

Note

Setting only before [bdb start](#).

nvram disable